Homework Assignment 5

Solution Manual

Question 1 Minimum spanning trees when costs change

(a) The MST has weight 14.

Kruskal's algorithm accepts AB of weight 2, CD of weight 3, CE of weight 4, and BC of weight 5. None creates a cycle; the four accepted edges connect all five vertices. Thus

$$T = \{AB, BC, CD, CE\}, \qquad w(T) = 2 + 5 + 3 + 4 = \boxed{14}.$$

For a direct justification, AB, CD, and CE are uniquely cheapest across the cuts that isolate A, D, and E, respectively. Also BC is uniquely cheapest across the cut $\{A, B\} \mid \{C, D, E\}$: its crossing weights are $5, 6, 7, 8$. The cut property forces all four edges into every MST, proving both minimality and uniqueness.

(b) A common increase preserves every MST.

There are four tree edges, so the new weight is $14 + 4(10) = \boxed{54}$. More generally, every spanning tree S of an n-vertex graph has $n - 1$ edges. If every edge weight increases by a constant c, then

$$w'(S) = \sum_{e \in S} (w(e) + c) = w(S) + c(n - 1).$$

Every spanning tree gains the same amount. Consequently $w'(S_1) \leq w'(S_2)$ if and only if $w(S_1) \leq w(S_2)$. Exactly the same trees minimize the total before and after the change. This includes negative values of c and the single-vertex case.

(c) The boundary is $x = 5$.

Among trees omitting AD, the unique cheapest one is T from (a), of weight 14. Those edge weights have not changed; the same cut arguments, with AD absent, still force its four edges.

A tree containing AD needs three more edges. The three cheapest available edges other than AD are AB, CD, CE, with weights $2, 3, 4$. They do form a tree together with AD:

$$T' = \{AB, AD, CD, CE\}, \qquad w(T') = \boxed{x + 9}.$$

No tree containing AD can have smaller weight, because its other three edge weights sum to at least 9. Equality forces precisely those three edges. Comparing $x + 9$ with 14 gives

$0 < x < 5$	T' is the unique MST
$x = 5$	T and T' are exactly the two MSTs
$x > 5$	T is the unique MST.

In particular, the tree from (a) remains minimum exactly when $x \geq 5$.

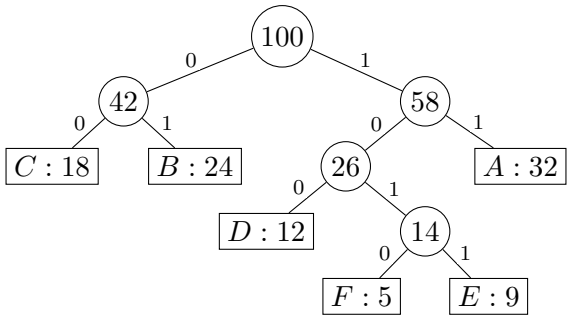
Question 2 Huffman coding and average length

(a) The merges and codewords.

Using probabilities as weights, the merges are

$$\begin{aligned} F + E &: 0.05 + 0.09 = 0.14, \\ D + (FE) &: 0.12 + 0.14 = 0.26, \\ C + B &: 0.18 + 0.24 = 0.42, \\ (DFE) + A &: 0.26 + 0.32 = 0.58, \\ (CB) + (DFEA) &: 0.42 + 0.58 = 1. \end{aligned}$$

The tree below uses percentages to keep its labels short.



s	A	B	C	D	E	F
$c(s)$	11	01	00	100	1011	1010
$\ell(s)$	2	2	2	3	4	4

(b) The expected length is 2.4 bits per symbol.

$$L = .32(2) + .24(2) + .18(2) + .12(3) + .09(4) + .05(4) = \boxed{2.4}.$$

For 1000 symbols the expected length is $\boxed{2400}$ bits. Linearity of expectation suffices; independence is not needed as long as each position has the stated distribution. A fixed-length code needs three bits per symbol: two bits give only four codewords, while three give eight. It therefore uses 3000 bits, so the expected saving is 600 bits, or 20%, excluding codebook overhead.

Question 2 Huffman coding and average length **Continued**

(c) Put the more likely symbol at the shallower leaf.

Swapping A and F changes their lengths from $2, 4$ to $4, 2$. All other contributions stay the same, so the new expected length is

$$L' = 2.4 + .32(4 - 2) + .05(2 - 4) = \boxed{2.94}.$$

In general, only the two exchanged symbols change their contributions:

$$L' - L = -\big(p(s) - p(t)\big)\big(\ell(s) - \ell(t)\big) < 0.$$

Both factors are positive. Exchanging symbols leaves the set of codewords unchanged, preserving prefix-freeness. Thus an optimal prefix-free code cannot give a strictly longer codeword to a strictly more likely symbol.

(d) An average is not a guarantee for each message.

The one-symbol message F uses four bits with Huffman but three with the fixed-length code. A message of m copies of F similarly uses $4m$ rather than $3m$ bits.

Expected length averages over the given distribution. For m symbols, the Huffman length has expectation $2.4m$, while the fixed-length code always uses $3m$. This does not promise a saving for every individual message.

Question 3 What happens when a tree loses a vertex?

(a) The components (including isolated vertices).

Deleted vertex	Number	Vertex sets of components
r	4	$\{a, e\},\ \{b, f\},\ \{c\},\ \{d\}$
a	2	$\{e\},\ \{r, b, c, d, f\}$
e	1	$\{r, a, b, c, d, f\}$.

(b) Each neighbor determines exactly one component.

Let the neighbors of v be $u_1, \ldots, u_d$. If two different neighbors u_i, u_j were connected after deleting v, a path between them avoiding v, together with $u_i - v - u_j$, would form a cycle in T. Therefore they lie in different components.

For any $z \neq v$, the unique path from v to z begins with some edge vu_i. Its remaining edges connect u_i to z after the deletion. Thus each component contains exactly one neighbor of v, giving exactly d components.

(c) Cycles can keep different neighbors connected.

In a triangle, deleting a degree-2 vertex leaves one connected edge, hence only one component. The tree proof relied on the absence of cycles: they would allow different neighbors to remain connected by a path avoiding v.

Question 4 Which roads must be retraced?

(a) The connecting road creates two odd degrees.

The degrees are

$$\deg(A) = \deg(B) = \deg(E) = \deg(F) = 2, \qquad \deg(C) = \deg(D) = 3.$$

The graph is connected, but the odd degrees at C, D rule out an Eulerian tour.

(b) The shortest closed route has length 8.

One route is

$$\boxed{A - B - C - D - E - F - D - C - A}.$$

It traverses AB, BC, DE, EF, FD, CA once each and CD twice, for a total of 8 edge traversals.

There are seven edges, so every route covering them must have length at least seven. To reach the right-hand triangle from A, the route must cross CD. To finish back at A, it must cross CD again: there is no other road between the two triangles. Thus a closed route must use CD at least twice and all six other edges at least once, giving the lower bound $2+6 = 8$. The displayed route attains this bound and is therefore shortest.

(c) A bridge must be crossed again to return.

Let $e = uv$ be a bridge in a connected graph. Deleting e separates the graph into two connected components, one containing u and one containing v. In the original graph, e is the only edge joining these two components; otherwise deleting it would not disconnect them.

A closed route that uses e must cross from one component to the other and eventually cross back to its starting component. Both crossings must use e. Thus every closed route covering all edges repeats e, whereas an Eulerian tour uses every edge exactly once. An Eulerian tour is therefore impossible.

(d) No bridges does not imply an Eulerian tour.

Take the complete graph on four vertices: join every pair of distinct vertices. Each vertex has degree 3, so it has no Eulerian tour. Nevertheless, it has no bridges. If any edge uv is deleted, a third vertex w still gives the path $u - w - v$. Any route that used uv can use this detour, so the graph remains connected.

The bridge condition is necessary but not sufficient. For a connected graph, the Eulerian-circuit theorem requires every vertex to have even degree.

Question 5 A consecutive block with divisible sum

(a) Repeated remainders locate a block.

For the given list, the partial sums and their remainders are

j	0	1	2	3	4	5	6	7
S_j	0	4	2	7	8	5	11	13
$S_j \bmod 7$	0	4	2	0	1	5	4	6.

For example, S_0 and S_3 both have remainder zero, giving the block $4, -2, 5$ with sum 7. Alternatively, S_1 and S_6 both have remainder 4, giving the block $-2, 5, 1, -3, 6$, also of sum 7. Either answer is sufficient.

(b) The general argument.

The objects are the $n+1$ partial sums $S_0, S_1, \ldots, S_n$, treated as indexed positions even if some values repeat. The n boxes are the residues $0, 1, \ldots, n-1$ modulo n. By the pigeonhole principle, two partial sums S_i, S_j have the same remainder. Order their indices so that $i < j$. Then

$$0 \equiv S_j - S_i = a_{i+1} + a_{i+2} + \cdots + a_j \pmod{n}.$$

This is a consecutive block, and it is nonempty because $i < j$. Every integer has a residue modulo n, so the reasoning allows negative terms, zeros, and repetitions. A block with sum zero is also valid, since $n \mid 0$.

Including S_0 is essential to this count: without it, we would have only n objects in n boxes, which does not guarantee a collision. It also allows the chosen block to start at a_1.

(c) One fewer entry need not suffice.

Take a list of $n-1$ copies of 1. Every nonempty consecutive block has a sum in $\{1, \ldots, n-1\}$, so none is divisible by n. This works for every $n \geq 2$, including the one-entry list when $n = 2$.